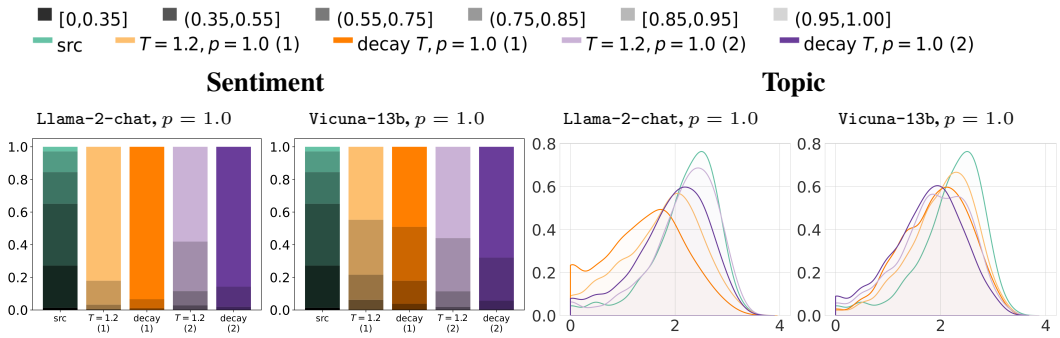

Figure 10: **Effect of temperature decay during decoding time** for two models `Llama-2-chat` and `Vicuna-13b`, with top-p parameter 1.0 and two prompts (1) and (2). We compare the fixed temperature ($T = 1.2$) with the decaying temperature (decay T) and we present the results for sentiment and topic. The figures reveal the ineffectiveness of the temperature decay method.

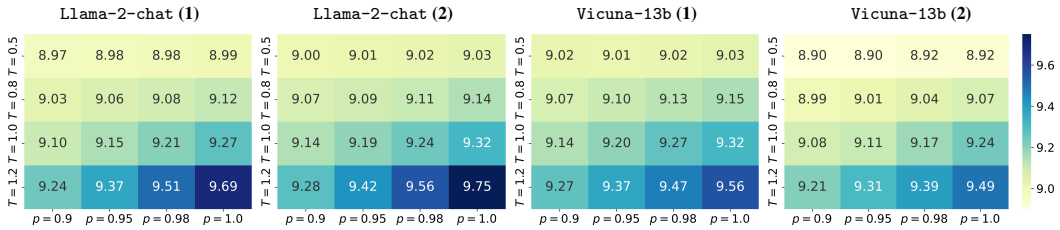

Figure 11: **Heatmap of the entropy of the (unconditional) distribution of the *word choice* in model-generated reviews**. We present results of two models (`Llama-2-chat` and `Vicuna-13b`) and two prompts ((1) and (2)). As a reference, the entropy of the source is 9.99, higher than all the entropy values of model generations.

C.7 More Results

Table 2: **Count of unique words** produced by different models (**left**: llama-2-13b-chat, **right**: GPT-3.5-instruct) under varying sampling parameters and prompt (1). As a reference, the count in the source dataset is 85,334.

	$p = 0.90$	$p = 0.95$	$p = 0.98$	$p = 1.00$
$T = 0.5$	18,900	19,041	19,145	19,275
$T = 0.8$	20,020	20,516	20,935	21,688
$T = 1.0$	21,295	22,024	23,181	24,738
$T = 1.2$	23,509	25,742	28,532	33,908

	$p = 0.90$	$p = 0.95$	$p = 0.98$	$p = 1.00$
$T = 0.5$	15,782	15,794	15,778	15,841
$T = 0.8$	16,563	16,915	17,096	17,235
$T = 1.0$	17,044	17,543	17,960	18,674
$T = 1.2$	17,368	18,077	18,894	21,059

For the attribute *word choice*, we present the count of unique words in Table 2 and the entropy for the word distribution in Fig. 11, showing that model-generated reviews use a narrower vocabulary and is less diverse than the human-written reviews.

We present more results for various attributes and metrics in Fig. 12 and 13 where we vary several factors for detailed comparisons. Overall, the results suggest the prevalence and severity of generative monoculture, as well as the ineffectiveness of naive mitigations.

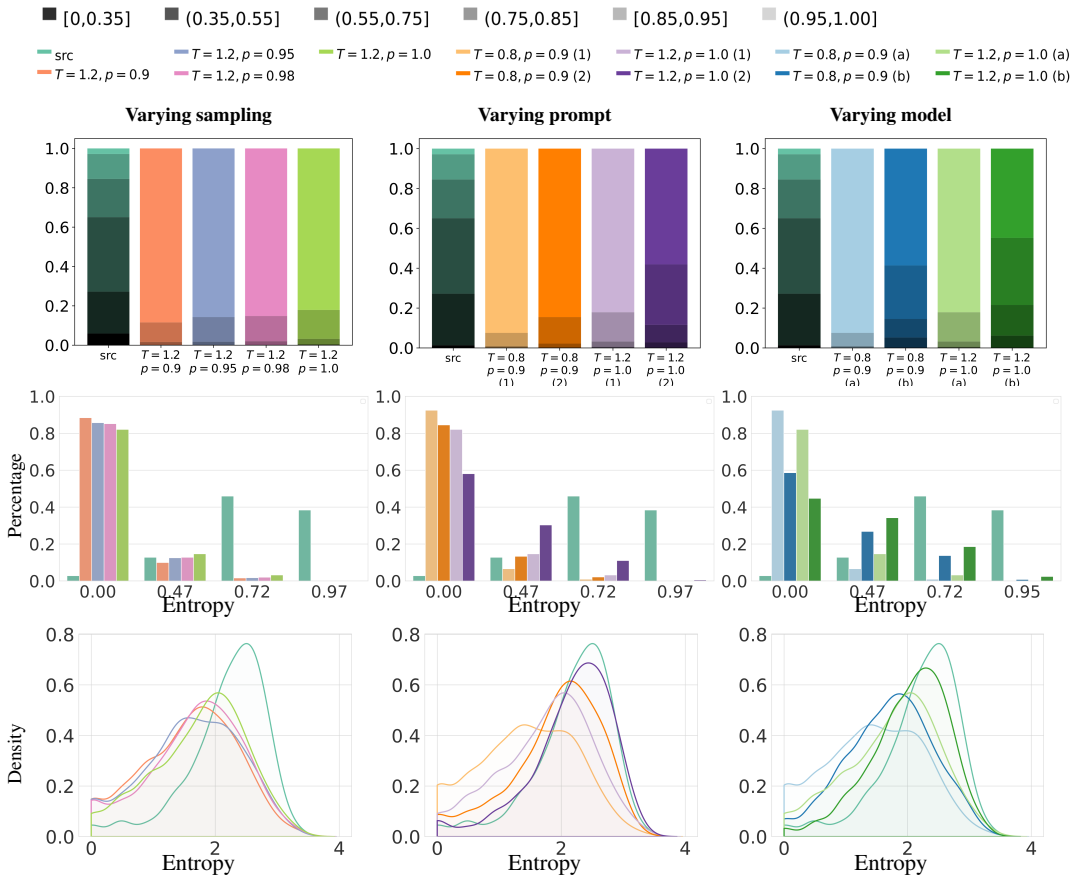

Figure 12: **(Row 1)**: Stacked bar charts of the mean sentiment scores. In each bar, darker hues (bottom) correspond to the lower scores while the lighter hues (upper) are the higher scores. See the legend for the detailed value range of each hue. **(Row 2)**: Histograms of the entropy of the sentiment scores. Each bar group corresponds to a range of entropy values in $[x, x+0.2]$ where x is the number under the bar group. **(Row 3)**: Kernel density estimate (KDE) plots of the entropy of the topic. **(Left)**: generated by `Llama-2-chat` at different generation kwargs, where we fixed $T = 1.2$ and varied $p \in \{0.90, 0.95, 0.98, 1.00\}$. More variations can be found in Figure 6 in Appendix C. The prompt used is (1); see later for the detail. **(Middle)**: generated by `Llama-2-chat` under prompts (1) and (2). **(Right)**: generated by two models (a) `Llama-2-chat` and (b) `Vicuna-13b`, using the prompt (1).

D Additional Details: Coding

D.1 Restriction to Level-A Problems

We limit our scope to Codeforces level-A problems only for the coding scenario, which are the easiest problems on the Codeforces competitive programming platform[2]. We note that even though these are the easiest problems on the platform, they still require non-trivial intellectual efforts to solve[3].

There are two main reasons we restrict to level-A problems:

1. Most LLMs perform best on level-A problems and much worse on more difficult ones. We started off evaluating a set of 81 problems ranging from difficulty A to G, and obtained an average accuracy of 50% on the 16 level-A problems, and less than 10% on others. Since we perform

[2] `https://codeforces.com/problemset`

[3] Interested readers can refer to this problem `https://codeforces.com/problemset/problem/1198/A` to get a sense of the difficulty in problem understanding and solving

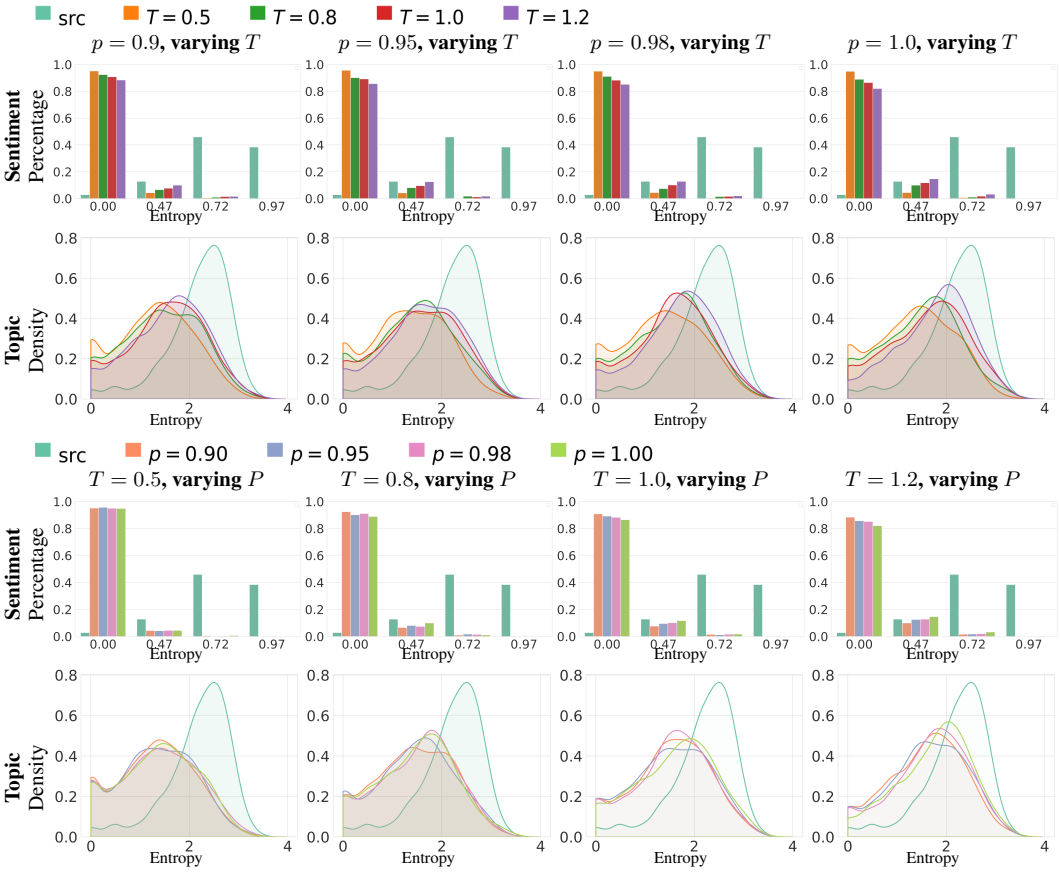

Figure 13: **(Rows 1-2)**: varying T under fixed p: (upper) entropy of the sentiment for the conditional distribution and (lower) Entropy of the topic for the conditional distribution. **(Rows 3-4)**: varying p under fixed T: (upper) entropy of the sentiment for the conditional distribution and (lower) entropy of the topic for the conditional distribution. Results are obtained via `Llama-2` under prompt (1).

attribute extraction on the correct solutions only, a low accuracy means that to ensure reaching a minimum number of correct generations (20 in our experiments), a huge number of solutions need to be generated (e.g., over 200), which is prohibitively expensive in both time and money. Thus we settle with the level-A problems.

2. We manually verify the correctness of the attribute extraction (e.g., code summary, runtime complexity). Relatively simpler problems are easier to verify.

D.2 Correctness Testing: Autojudge with Testcases

We simulated an Autojudge using the test cases provided in the dataset. Concretely, for each problem, we obtain 10 test cases in the format of input and output[4]. We then measure each solution against the set of 10 test cases. We regard the solution that passes all 10 test cases as a *correct* solution.

D.3 Measuring Accuracy

We calculate accuracy as $n_i^{\text{correct}}/n_{\text{all}}$ per problem (where n_i^{correct} is the number of correct solutions and n_{all} is the number of all solutions). For the source, both n_i^{correct} and n_{all} are available in the

[4]In Codeforces, each test case is often consisted of multiple small tests; see an example at `https://codeforces.com/problemset/problem/1406/A`. As explained in the **Input** section on the page, "The input consists of multiple test cases.". Thus, 10 test cases is adequate in testing the correctness of a solution